

A continuous rung host, version 3

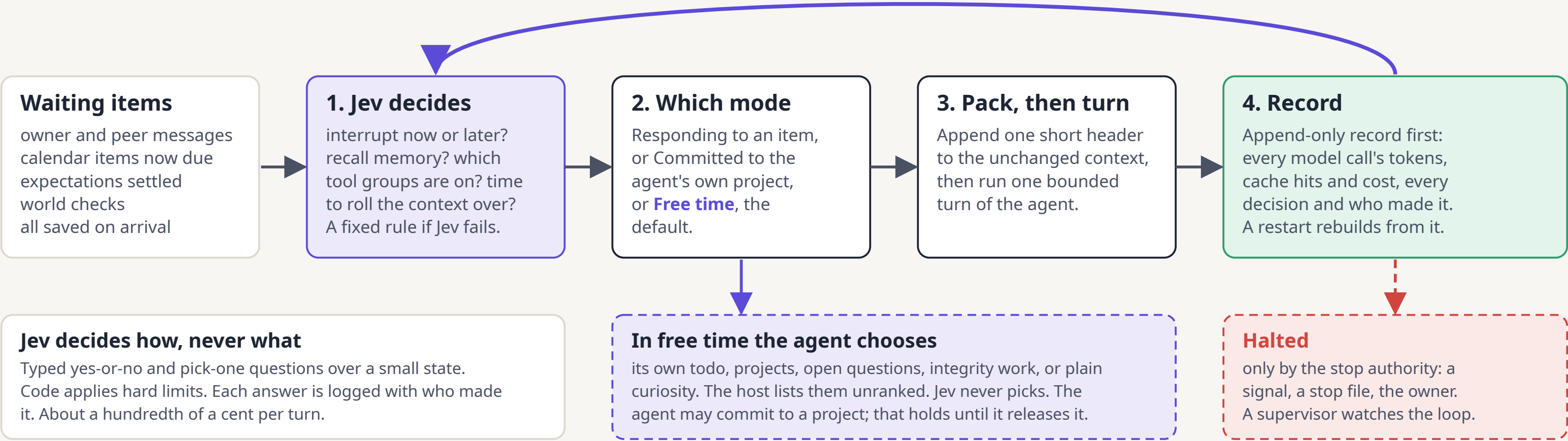
The agent owns its own time. A small decision model, Jev, runs the host's machinery. The context is built so a provider's prompt cache can reuse almost all of it.

1. **No more host-side drives.** Version 2 scored the agent's pulls and picked one. That is gone. **Free time is the default:** when nothing has arrived and nothing is committed, the agent decides what to do, and it may commit to a project.
2. **Jev runs the machinery.** Typed questions decide whether an item interrupts, what to inject, which tools are on, and when to roll the context over and save memory. Each decision has a fixed rule behind it. Cost: about a hundredth of a cent per turn.
3. **Context is a cache design.** Stable parts first, slow parts next, then a log that is only ever appended to. Tools stay defined and are switched on or off by the host.
4. **Free models first.** The named stealth model exists and is free, but it expires in two days. A ladder of free models follows it, and on free models the request quota per day, not money, sets the pace.
5. **One agent first.** Workers and delegation come last, as an extension. Three questions for the owner are on the last page.

One loop, three modes, and the agent picks what to do with its time

Jev decides how the host behaves at each boundary. The agent decides what it wants. The loop still never rests.

Again, at once. The only waits are ones the world forces, such as a free-tier quota.

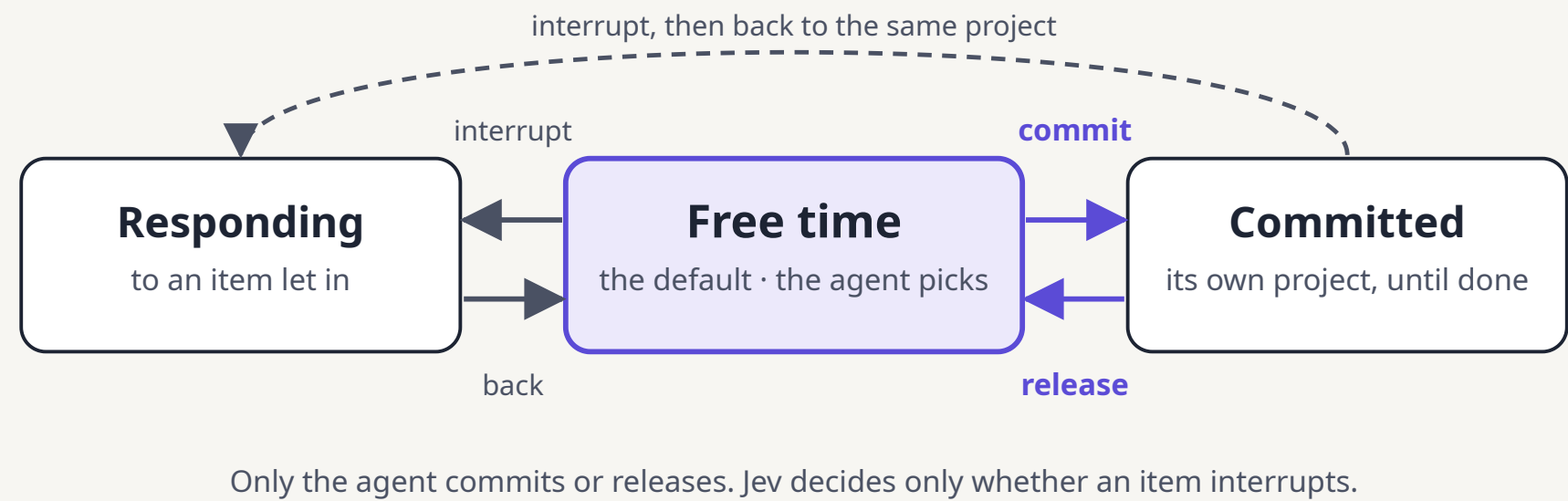


CHANGED from version 2

The scored pulls, the ballot, fairness floors and forced integrity turns are **dropped** · free time and the agent's own commit and release are **new** · Jev now makes the host's mechanical decisions · the context order is **rebuilt** for caching: version 2 put the changing status block near the top and slid a window of recent turns, which spoiled the cache every turn · the first live run uses **free hosted models** · workers move to a **final extension** · two new pass-or-fail gates: cache discipline and decisions.

When nothing is waiting, the agent's time is its own

An operating system runs an idle process when nothing else is ready. Here the idle process is free time, and what fills it is the agent's choice.



What the host shows in free time

- Once, in the stable part of the context: the free-time rules, adapted from a reference agent's curiosity practice. No obligation. It has to be your own pull. Follow it. Leave a trace. Keep what you found.
- Each turn, a short header: the time, how long since anything arrived, the quota left, and the agent's own material: todo, projects, open questions, integrity facts, recent traces.
- The material is **listed by creation date, never ranked**. A test checks that no score can change the order.

What interrupts it

- Jev judges whether a waiting item is worth interrupting the current mode now, later, or as a one-line note.
- Hard limits: an owner message always gets through. Each kind of item has a maximum wait. A commitment is interrupted at most six times an hour.
- If the agent starts copying itself, the host rolls the context over early and tells the owner. It changes the context, never the topic.

FIRST PROJECT The owner seeds one project: build its own anticipatory model of itself and its world. It is one option among the agent's own. The expectation register and the record of the host's own behaviour are its raw material.

Five decisions, each a typed question with a rule behind it

Jev answers yes-or-no and pick-one questions over a small state the host builds. Code combines the answers and applies hard limits. If Jev is unavailable, slow or over its cap, a fixed rule decides.

DECISION	QUESTION SHAPE	INPUTS (BOUNDED)	CHOICES	RULE WITHOUT A MODEL, AND HARD LIMITS
Admit	"Should this waiting item be shown now, displacing the planned continuation, or wait for the agent's next break?"	mode; up to 8 items with kind, role, age, a short gist; interruptions in the last hour	now · at the break · one-line note	Owner messages and firm calendar items now; others at the break. A maximum wait for each kind; at most 6 interruptions of a commitment an hour
Inject	"Would one block of recalled memory help this turn?" and "Which text should be the cue?"	what was let in, or the commitment's goal; when memory was last recalled; counts of items due soon	recall or not · cue · optional summaries	Recall on every reply and at the start of a commitment; otherwise every tenth free-time turn. Everything injected goes in the newest turn header only
Tools	"Should this tool group be callable in the turn about to run?"	mode; groups now on; the owner's ceiling; the agent's requests with reasons; recent use	on or off for each group	Core, memory and reading always on. Never beyond the ceiling. A group stays on for at least 3 turns
Pack	"Roll the context over now, at the next break, or keep appending?" and, per passage, "Will the exact text be needed next epoch?"	context size against its budget; at a break or not; up to 10 passages with gists	append · roll over now · at the break; keep or summarise	Roll over at the first break past 60%, always at 85%, never below 40%. Keep what open work refers to
Consolidate	"Should the agent be asked to update its note first?" and, per candidate, "Is this worth keeping in long-term memory?"	note age; events since; up to 10 candidates	ask or not; keep or not	Keep every checked turn and settled outcome; offer a note update before every rollover. The agent writes the note, never Jev

JEV DECIDES HOW

When to show something, what to add to the context, which tools to expose, when to compact and what to keep word for word. Always inside limits written in code, and every answer is recorded with who made it.

JEV NEVER DECIDES WHAT

What the agent does with free time, which project it takes, whether it commits or releases, the content of anything it writes, the tool ceiling, settling expectations, or stopping the host.

Cheap, fast, logged, and tested without the network

One combined question set per turn covers admit, inject and tools. Pack and consolidate are asked only when the context nears its budget or the agent reaches a break.

Cost

ITEM	VALUE
Price, verified	\$0.042 per million input tokens; output free
Per turn, estimated	about 2,200 tokens: about \$0.0001
Per rollover	about 1,800 tokens: under \$0.0001
Per day, free tier	about 250 turns: about 2.5 cents
Per day, local model	about 2,000 turns: about 20 cents
Default caps	25 cents a day, a tenth of a cent a turn

Token counts use rung's own estimator, which runs deliberately high.

Latency

- Measured earlier on smaller states: half of calls under 0.2 s, 95% under 0.3 s.
- Budget: 0.8 s per turn, cut at 2 s, then the rule decides.
- A free model's turn takes seconds, so this is under a tenth of the turn.

Every decision is recorded

The record holds the question set, the answers, the choice, and who made it: Jev, with its cost and time, or the rule, with the reason (no decider, cap reached, too slow, or an unusable answer).

Tested offline first

- A scripted decider drives every decision and every failure path in the first slice, at no cost.
- Recorded exchanges replay real answers. Rewording a question fails the replay until it is re-recorded.
- **Shadow week** on the first live run: the rule decides, Jev is asked and logged, and disagreements are audited.
- A decision family switches to Jev only when a test criterion written in advance holds.

UNKNOWN How well Jev judges these mechanical questions has not been measured.

What Jev sees

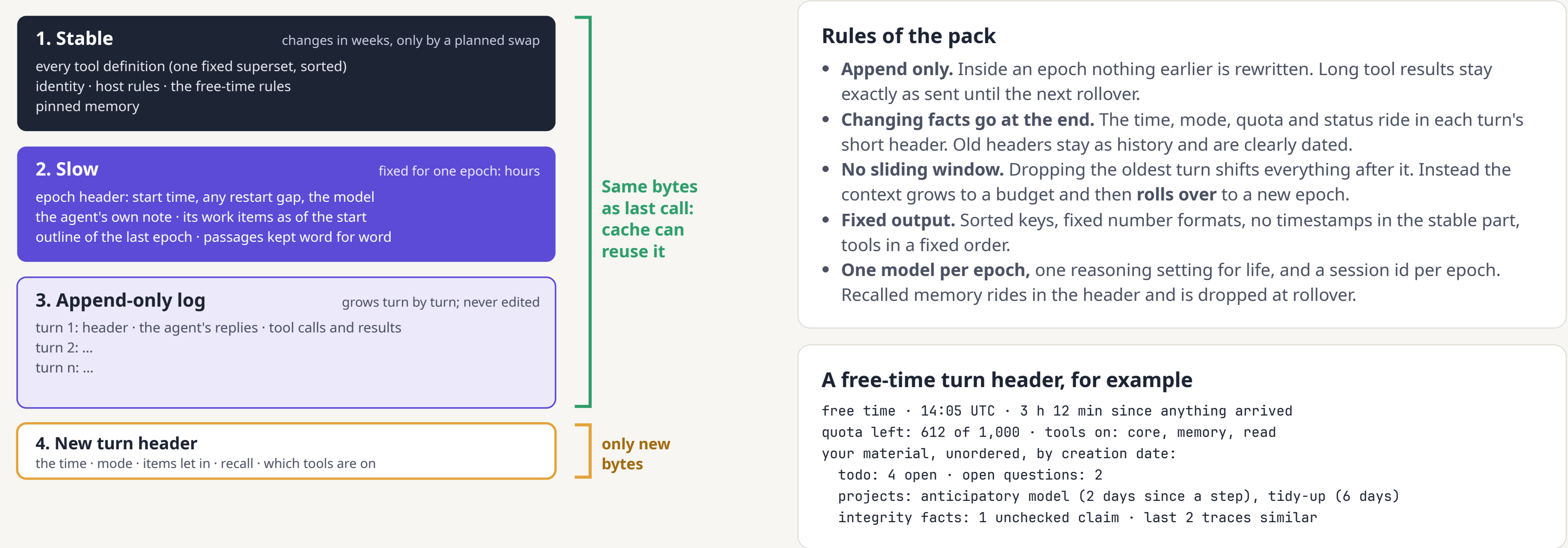
A small state built in code: short gists of at most 200 characters, counts, ages and named options. Never raw tool output, never credentials. Gists pass the existing redaction first. A state too large for Jev's 32 thousand token window is refused before sending, never cut.

How the answers are used

Each question is atomic. Code combines the answers with thresholds that are settings, not guesses buried in prompts, and the hard limits always win. An unusable answer is never read as a favourable one: the rule decides instead, and the record says why.

Build the context so almost all of it repeats byte for byte

A provider's prompt cache reuses its work only on an identical opening stretch of the prompt. So what never changes goes first, then what changes slowly, and each turn only adds at the end.



What spoils the cache, and how tools are switched without spoiling it

Tool definitions sit at the very front of the prompt, so changing them throws away everything. The host keeps one fixed set and controls which tools may be called.

What spoils it

CHANGE	EFFECT AND POLICY
Tool definitions	Everything is lost. Change only in a planned swap at a rollover
Model or provider	Everything (a different cache). One model per epoch
Identity, rules, pinned memory	All but the tools. Queue edits for a swap
Tool choice setting	One vendor: the conversation part is lost. Never vary it per turn
Reasoning effort	Can be lost. Pin one setting
Rewritten history	Lost from the edit on. Never rewrite inside an epoch
Status block near the top	Lost every turn. Facts go in the newest header
Idle longer than the cache lives	Lost. Typically 5 to 30 minutes; the router's sticky session lapses after 10. Recorded as a cold cache, not a fault

Four ways to switch tools

WAY	CACHE · WHERE IT WORKS · VERDICT
Fixed set, host gate	Every definition is sent every time. Calling a disabled tool returns "not enabled this turn; ask for it", and nothing runs. Cache untouched, works everywhere , including free models that only allow automatic tool choice. Default
Allowed-tools setting	One vendor recommends it and keeps its cache; another vendor loses the conversation cache when it changes. Optional, per provider
Router tool search	Keeps the cache, but only on two newer interfaces, not the chat interface rung uses. Later, if tools grow past about ten thousand tokens
Swap the list	Everything is lost. Rare, batched with a rollover

Risk of the gate: a model keeps calling disabled tools. An experiment counts it; above 5% of calls, add the allowed-tools setting where it is safe.

The router's part

The router keeps a conversation on the same provider so its cache stays warm, for up to 10 idle minutes. A session id makes this hold from the first call; the host uses the epoch id. It applies only where cached reads are cheaper than normal ones, so it does nothing on free models. The router can also fall back to other models by itself, but the host walks its own ladder so that every turn records which model answered.

Roll over rarely, measure every call, and test cheaply

Compaction becomes a planned rollover that leaves the stable part alone. Every model call's cache numbers go into the record, so a broken prefix shows up at once.

The rollover

1. Jev picks the moment inside fixed limits, at a natural break when it can.
2. Memory first: the outcomes are kept, and the agent is offered one line to update its note.
3. A new slow part from the record: the latest note, the work items, a mechanical outline of the last epoch, passages kept word for word.
4. No model-written summary. The record stays the truth; the agent's synthesis is its own note.
5. The stable part is untouched, unless a planned swap is batched in here.

Measured on every call

- The record keeps tokens sent, tokens read from cache, tokens written to cache, reasoning tokens, cost, latency, and the model and provider that served it.
- **Hit rate** is cached over sent. **Efficiency** is cached over what should have been cached, which the host knows.
- When efficiency drops, the host compares fingerprints of the stable and slow parts. A change it did not plan is a bug, and a test gate fails on it.
- At present the library drops cost and cache-write counts on one response path. A small fix.

VERIFIED

Free models have no cache

Every free endpoint listed reports no caching and no cache price. On free models this work buys nothing measurable yet. It pays off on a local model, where reused prefixes cut waiting time, and on any paid fallback.

Cheap experiments, designed only

- Does any free route report cache hits? Two identical requests per model, free.
- Version 3 pack against version 2 order on a cheap caching model, a few cents, with the owner's word.
- How often models call disabled tools, free.
- A local model's response time against context length.

Known and unknown, by provider

ROUTE	HOW IT CACHES	CACHED READS COST	HOW LONG IT LIVES
Free models on the router	none advertised, on every one listed	no cache price	unknown whether any reports hits
Two large vendors	automatic over a minimum size, or marked break points	a tenth to a half of normal	5 minutes to an hour; at least 30 minutes for one
Several other vendors	automatic	a tenth to a half of normal	minutes; varies
A local model server	reuses a shared opening stretch	no money; saves waiting time	while memory allows; inferred, not read

The named stealth model is real and free, but it expires in two days

Checked on the router's public catalogue. The ladder after it uses free models with tool calls, long context and good uptime. On the free tier, the request quota sets the pace.

VERIFIED

space-bunny-alpha

Price	free in and out
Context	1 million tokens; up to 524 thousand out
Tools	yes, automatic choice only; tools cannot be switched off by setting
Reasoning	always on; default effort maximum
Caching	none
Uptime	99.9% over one day; one provider
Expires	5 October 2026
Unknown	its rate limit; whether prompts are logged; quality on rung's work

Quota, verified in the router's limits

Free models: 20 requests a minute; 1,000 a day if the account ever bought 10 credits, otherwise 50. A turn makes about 3 to 6 model calls, so 1,000 allows **about 250 turns a day**. At 50 the free tier cannot host the loop. The host spreads the quota over the day, keeps a quarter for replies, and records waits as quota waits, not rest.

The fallback ladder

#	MODEL	WHY
0	stealth / space-bunny-alpha	the owner's pick, only while listed
1	nemotron-3-ultra 550B, free	largest free tool model; 1 million context
2	qwen3.8 27B, free	agent-capable family; 262 thousand
3	gemma-4 31B, free	full tool control; 99.7% uptime
4	nemotron-3-super 120B, free	a different vendor from rung 3
5	the router's random free model	last resort; recorded as random

Backoff

- The catalogue is re-read at start and every 6 hours; a model that is gone, expired or degraded is skipped.
- Provider overload, errors or timeouts: step down a rung for 2 minutes, doubling to 30, then try back up.
- The router's own quota limit: wait for its reset time. Switching models does not help, because the quota counts the whole account.
- Stay on one model per epoch, and say so when it changes. No paid fallback without the owner's word.

Most of version 2 carries over; delegation comes last

The library split, the outward protocol, memory, the record and the stop all stand. Workers are kept as an extension point only.

Crates and the library

- A new host crate runs the loop; the agent becomes a library first.
- **Owner's rule:** the library is a new core crate, and the current agent binary becomes a thin shell over it. Same name, flags, protocol, memory extension, exit codes. The existing tests pass unchanged, a recorded session plays the same through old and new, and a pinned consumer still works.
- Four new small gaps: cost and cache numbers per call, no rewriting of saved history, a per-turn tool gate, and a session id for the router.

Unchanged from version 2

- One agent, many protocol channels; each prompt is saved on arrival and gets exactly one answer.
- The host owns memory; one recall block per turn.
- Time is context; one host calendar; no wake-me tool.
- The expectation register: the host settles, never the agent. Surprise is recorded, but gives no bonus.
- Stop from a signal, a stop file or the owner; a supervisor and watchdog outside the loop.
- Append-only record; a restart rebuilds everything, states the gap, and starts a fresh epoch.
- Provider trouble means backoff, never death.

Delegation, last

- Slices 0 to 3 are **one host and one agent**, end to end.
- Kept now: an extension point (worker results would arrive as items that Jev judges like any other), a reserved tool-group name, and record kinds.
- Kept now: the **responsiveness bound**. A turn has a time limit; long work is refused with "break it into steps, or commit to it as a project".
- Later, in order: workers with host-checked results; workers side by side in one process; world effects beyond the sandbox, only through workers.

Provider trouble: back off, never die

TROUBLE	WHAT THE HOST DOES
Provider overload, errors, timeouts	Return the items to the head of the line, step down the ladder, wait with growing delays up to 15 minutes
The router's quota	Wait for its reset; record the wait as a quota wait
Bad key, no credit	Mark the host blocked, tell the owner once, retry every 15 minutes, keep accepting items
Refusal, or context too long	Record it and go on; roll the context over if it is too long

Safety valves, not budgets

VALVE	RULE
Quota pacer	Spreads the free requests over the day; a quarter kept for replies; at most 6 model calls per turn
Jev cap	25 cents a day; past it, the rules decide until the next day
Spending cap	Only if a paid model is ever allowed; it halts the host
Copy guard	Three copied turns in a row: early rollover and a note to the owner

Prove the single-agent host at no cost, behind frozen gates

Slice 0 splits the library with no change in behaviour. Slice 1 builds the host against a scripted model, a scripted decider, a fake world and injected faults.

Twelve gates

GATE	PASSES WHEN
No rest	Over 30 quiet minutes, under 2% of the time is outside a turn, apart from declared waits; every turn's decisions are recorded
Responsive	Owner messages get in within one turn's duration; long work is refused
Free time NEW	With nothing waiting and nothing committed, every turn is free time; a commitment holds until release; no score can change the order of the agent's material; only the agent's tools can commit or release
Schedule	Due items enter at the next boundary; items missed while down fire once, marked late
Expectations	Only the host settles; calibration recomputed from the record matches
Provider faults	Never exits; requeued items enter once; the ladder steps down on provider limits and waits on quota limits
Stop	Halts within a second from any wait; the watchdog catches a wedged loop
Hard kill	After 50 random kills each item has one outcome; state and mode rebuild
Bounded	Over 10,000 turns the context never passes its budget
Cache NEW	Inside an epoch each request starts with the previous one, byte for byte; the stable and slow fingerprints change only at a planned swap or rollover
Decisions NEW	Every decision falls back cleanly on every failure, within 2 s; hard limits hold against adversarial answers
Cost	Zero: no network, no live model, no live Jev

Main risks

- **Rumination:** with no host arbiter nothing forces breadth. The copy guard and early rollover are untested on a real model.
- **Jev misjudges:** mitigated by hard limits, rules and a shadow week.
- **Free-tier starvation:** the quota keeps the loop waiting most of the time.
- **Model churn:** the stealth model expires; switches change the voice and are always stated.
- **Disabled-tool calls** waste steps; measured.
- **Exposure** of an always-on endpoint: loopback by default, role tokens, owner-only stop.

Size, inferred: slice 0 about 1,000 lines, slice 1 about 4,000 to 5,000 with tests. Slice 2 adds the protocol and the real engine; slice 3 is the live run on the free ladder.

Three questions, most blocking first

The same three as version 2, with recommendations updated for version 3. Everything else I decided; those choices are listed so any can be overruled.

1. Build slices 0 and 1 now?

May the library be split, with the agent binary kept as a thin shell, and the single-agent host then built against a scripted model and a scripted decider, at no cost and behind the gates?

Recommendation. Yes. Both are free, reversible and gated, and nothing reaches a live model or live Jev.

What the answer changes. Whether the held first-slice task is released to build, or the design waits.

2. First live run: a free hosted model now, a local GPU later?

Which substrate hosts the first real run?

Recommendation. Yes: the free ladder now, starting with the stealth model only while it is listed, then the Nemotron Ultra, Qwen, Gemma and Nemotron Super free models. Jev runs accounted, capped at 25 cents a day, in shadow mode for the first week. No paid fallback. Move to a local GPU when the free quota proves binding.

What the answer changes. The live substrate, whether the account needs the 10-credit tier, and whether the cache work pays now or later.

3. May the agent change the world itself, or only through workers?

Should the always-on agent hold write and shell tools itself?

Recommendation. Itself, but only inside a sandboxed workspace that the host owns. Every effect beyond it waits for host-checked workers in the extension. Jev may switch tools off within that ceiling, never on beyond it.

What the answer changes. The tool ceiling, the safety envelope of an agent no one watches, and what the worker extension must carry.

Decided by me

- Owner messages always interrupt; Jev judges all other items.
- One commitment at a time; the host never times one out.
- Free-time material listed by creation date, never ranked.
- Five decision families, one combined question set per turn, caps of 25 cents a day and a tenth of a cent a turn, shadow mode first.
- A fixed tool set with a host gate; swaps only at a rollover.
- Epoch budget: 128 thousand tokens or half the window, whichever is less; roll over between 40% and 85%; no sliding window; no model-written summary.
- One reasoning setting, one model per epoch, a session id per epoch.
- A quarter of the free quota kept for replies; at most 6 model calls per turn.
- Copy loops cause an early rollover and a note to the owner.
- Delegation deferred to the final extension.